

Labyrinthian - Technical Documentation

Author: Vojtěch Jersák

Date: May 20, 2026

1. Basic information

- **Name:** Labyrinthian
- **Author:** Vojtěch Jersák
- **Software:** Java 25, Javafx 21
- **Build tool:** Maven
- **IDE:** IntelliJ IDEA
- **Gitlab reposirory:** https://gitlab.fel.cvut.cz/B252_B0B36PJV/jersavoj

2. Architectural Design Patterns

The system relies heavily on decoupling core engine updates from rendering structures to maximize stability, performance, and clear data ownership boundaries.

2.1 Model-View-Controller (MVC) Pattern

- **Model:** This stores the game's data and logic. It keeps track of things like the map layout, the player's health, inventory, and object positions. It runs separately from the graphics, which makes it easy to test the game's math and rules on their own.
- **View:** This is what the player sees. It includes the user interface (built with FXML), the game window, and all the images, textures, and sprites used to draw the game.
- **Controller:** This acts as the middleman. It listens for player input (like keyboard presses or mouse clicks), updates the game's data (the Model) based on those inputs, and then tells the graphics (the View) what to draw next.

2.2 Game Loop Integration

The game runs continuously using JavaFX's AnimationTimer. This loop updates 60 times per second (60 FPS) to keep everything running smoothly. Each singular frame loop cycle parses through three processing stages:

1. **Input Collection:** Has a separate set for movement keys, allowing multiple keys pressed at once for smooth omni-directional movement. The rest of key inputs are handled as one press active at a time.

2. **Update:** Moves player movement based on inputs, updates all entities (movement, state machine, attacking...) calls the renderer to update all visuals.
3. **Rendering:** Redraws the whole map with updated entity positions, moves “camera” so that player is always in the middle of the screen, updates sprites based in relevant events.

3. Game Information & Controls

All hardware inputs are intercepted by the scene's event filters and passed to the InputManager for state tracking.

Key Input	Action Description
W	Move forwards
S	Move backwards
A	Move left
D	Move right
Space	Attack
1 - 5	Select active inventory slot
Q	Use currently active item
E	Interact / pickup item / talk to NPC / transform BOSS
F	Delete item in the current active slot
ESC	Pause game

4. Core Game States

Game states upon which the state machine operates.

State Enum	Description
MAIN MENU	The main menu, player can start a new game or load a saved one.
RUNNING	The game is active, the player moves through the maze, enemies are active.
PAUSED	Game paused via ESC, a menu is shown with save and quit options.
DIALOGUE	A dialog window is displayed (item description or BossNPC conversation), game is paused.
LEVEL COMPLETE	Player passed through the portal, transition to next level, health is restored.
WON	Player completed all 5 levels, stats are shown.
GAME OVER	Player died, transition to game over screen.
SHOWING END	Showing game over screen with stats.

5. Class Descriptions & Component Registry

The project is decomposed into multiple classes across relevant packages

5.1 Core Package

Class Name	Package	Description
GameApplication	Core	Entry point of the application. Initialises the JavaFX window, Canvas, Scene and AnimationTimer (game loop). Creates GameManager and Renderer.
GameManager	Core	Central class managing the game state.

Class Name	Package	Description
Renderer	Core	Draws the map tiles, player and enemies onto the Canvas.
GameTimerService	Core	Creates a task running on separate thread to track in game time.
InputManager	Core	Processes keyboard inputs.
SaveManager	Core	Menu for saving and loading game state.
GameState	Core	Enum defining all possible game states.
Launcher	Core	Launches the application.
Save data	Core	Data class for game saving
Save manager	Core	Saves and loads the game using the json system

5.2 Entities Package

Class Name	Type	Description
GameObject	Abstract	Abstract base class for all objects placed in the world.
Entity	Abstract	Abstract class extending GameObject. Adds health (maxHealth, currHealth), takeDamage(), isDead(), move() with collision detection, and abstract onDeath(GameManager).
Player	Class	The player character. Extends Entity. Holds an Inventory and an active Weapon (default: Sword). onDeath() triggers GAME_OVER.

Class Name	Type	Description
Enemy	Class	Basic enemy. Extends Entity with baseDamage, attackSpeed and attackRange.
Boss	Class	Powerful enemy extending Enemy. Supports multiple attack types (Attack Types enum). Can be transformed into a friendly NPC via transform().
ClayPot	Class	Breakable object extending Entity (maxHealth = 3). Holds an Item; onDeath() drops the item at the pot's location.
Projectile	Class	Class for the projectiles shot by the boss during a ranged attack
AttackTypes	Enum	Enum of boss attack types: MELEE, RANGED, AOE.
Directions	Enum	Enum of cardinal directions and relevant sprite indexes.
EnemyState	Enum	Enum of possible enemy states for the enemy AI state machine.

5.3 Items Package

Class Name	Package	Description
Item	Items	Abstract base class for all items. Implements Interactable. Stores name and description. Declares abstract use(Player, GameManager).
Consumable	Items.Consumables	Abstract subclass of Item for consumable items. Adds numberOfUses, decreaseUses() and usedUp(). Implements onInteraction() to add the item to the player's inventory.

Class Name	Package	Description
BlindingStew	Items.Consumables	Applies a random positive effect (e.g. +3 hearts, 2x damage, speed boost) but reduces the player's field of view. Effects last until end of level.
SnickersBar	Items.Consumables	Restores 2 hearts. If used near the boss, has a 10% chance to transform the boss into a friendly NPC, allowing the player to pass freely.
RustyPogoStick	Items.Consumables	Allows jumping over a hedge wall if the tile behind is free. Deals 1 heart of self-damage on landing. Unlimited uses (numberOfUses = -1).
YarnBall	Items.Consumables	When activated, leaves a visible trail behind the player showing the path taken.
CO2Laser	Items.Consumables	Permanently converts one hedge tile into a path tile. Lasts 3 uses.
CO2Shears	Items.Consumables	Same effect as CO2Laser but single-use (1 use).
Weapon	Items.Weapon	Abstract subclass of Item for weapons. Adds damage, attackSpeed and range. Declares abstract attack(Entity).
Sword	Items.Weapon	The player's starting weapon. Not obtainable as loot. Stats: damage 1, attackSpeed 1, range 1.
UltimateObliterator	Items.Weapon	Rare weapon that destroys any enemy in one hit. Equipping it permanently reduces the player's max health to 1. Cannot be unequipped once picked up.

5.4 World & Components Package

Class Name	Package	Description
TileType	World	Enum defining tile types: PATH (walkable) and HEDGE (impassable hedge/wall).
Tile	World	Represents a single map tile. Stores a TileType and boolean isWalkable().
Map	World	A 2D array of Tile objects (Tile[][]).
WorldBuilder	World	Generates the map (buildMap()) and places enemies, claypots and boss.
EscapePortal	World	The portal to the next level. onInteraction() triggers LEVEL_COMPLETE.
Interactable	Components	Interface with a single method onInteraction (Player, GameManager). Implemented by Item, EscapePortal and Boss.
Inventory	Components	The player's inventory with a capacity of 5 slots for consumables and 1 separate weapon slot.
Utils	Components	Utility class with methods shared across the entire project (e.g. distance calculator)
GameStats	Components	Keeps track of players statistic such as score, level time and average level time

5.5 UI Package

Class Name	Description
MainMenuScreen	Displays the main menu using a JavaFX Scene with a VBox layout.
PauseMenuScreen	Displays the pause menu as a JavaFX Scene with a VBox layout.
EndScreen	Displays the winning / game over screen with run statistics as a JavaFX scene
DialogueScreen	Renders a dialogue box over the game canvas, e.g. item description after pickup or BossNPC conversation.

Class diagram:



